

# Day 01: Introduction to Java

# Introduction to Java

## 1. What is Java?

Java is a high-level, object-oriented, class-based and platform-independent programming language developed to build a wide variety of applications.

Java programs are compiled into bytecode, which is executed by the JVM (Java Virtual Machine).

## Java Source Code



javac



## Bytecode



## JVM



## Output

In short: Java is a platform-independent, object-oriented programming language used to develop different types of applications.

## 2. Usage of Java – Where is Java Used?

Java is widely used in many areas:

In short: Java is widely used for enterprise, web, mobile, cloud, Big Data, microservices, desktop and distributed applications.

### 3. Components of Java

The three important terms are JDK, JRE and JVM.

## JDK

[illegible]

11

## Runtime Development

## Components Tools



## JVM

JDK – Java Development Kit

JDK is used to develop Java applications. It provides tools such as javac, java, and jar.

JDK → Develop

JRE – Java Runtime Environment

JRE provides the runtime environment and libraries required to run Java applications.

JRE → Run

JVM – Java Virtual Machine

JVM loads and executes Java bytecode.

JVM → Execute

JDK → Develop | JRE → Run | JVM → Execute

## 4. How C++ Works

C++ is traditionally compiled into native machine code for a particular platform.

C++ Source Code

↓

Preprocessor

↓

Compiler

↓

Object Code

↓

Linker

↓

Executable File

↓

## CPU

For example:

hello.cpp

↓

Compiler

↓

hello.exe

↓

## CPU

In short:

C++ → Compiler → Native

## 6. How Java Works

Java follows a compile and execute process. First, the Java source code is compiled by the Java compiler (javac) into bytecode. The bytecode is stored in a .class file. The JVM then loads and executes the bytecode using interpretation and JIT (Just-In-Time) compilation.

Java Source Code

(.java)

↓

javac

↓

Java Bytecode

(.class)

↓

## JVM

↓

Interpreter + JIT

↓

Native Machine Code

↓

## CPU

In short:

Java → javac → Bytecode → JVM → Machine Code → CPU

This is what enables Java's platform independence.

## 7. Features of Java

### 1. Simple

Java is relatively easy to learn and avoids complex features such as direct pointers.

### 2. Object-Oriented

Java organizes programs using classes and objects.

### **3. Platform Independent**

Java bytecode can run on different operating systems using a compatible JVM.

### **4. Portable**

Java programs can be moved between different platforms with minimal changes.

### **5. Secure**

Java provides bytecode verification, type checking, access control and controlled memory management.

### **6. Robust**

Java provides strong type checking, exception handling and automatic memory management.

### **7. Multithreaded**

Java supports executing multiple tasks concurrently using threads.

### **8. Distributed**

Java provides networking and web-service APIs for applications that communicate across different computers.

### **9. High Performance**

Java uses JIT compilation to improve execution performance.

### **10. Dynamic**

Java can load classes and libraries at runtime when required.

#### **11. Architecture Neutral**

Java bytecode is not tied to a particular processor architecture.

#### **12. Automatic Garbage Collection**

Java automatically removes objects that are no longer required from memory.

## **8. Java Environment**

To develop and run Java programs, we need a Java development environment.

Required Software

### **1. Operating System**

Examples:

Windows

Linux

macOS

## 2. JDK (Java Development Kit)

The JDK provides the tools required to develop Java programs.

Examples:

Oracle JDK

Eclipse Temurin

Amazon Corretto

## 3. IDE (Integrated Development Environment)

An IDE makes Java development easier.

Examples:

Eclipse

IntelliJ IDEA

Visual Studio Code

NetBeans

Basic Environment

Operating System

↓

**JDK**

↓

**IDE**

↓

Java Program

Verify Installation

Open Command Prompt/Terminal and execute:

java -version

and:

javac -version

## 9. Hello World Program in Java

The simplest Java program is:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello World");  
  
    }  
}
```

```
}
```

## Output

Hello World

## Explanation

```
public class HelloWorld
Defines a class named HelloWorld.
public static void main(String[] args)
The standard entry point of a traditional Java application.
System.out.println()
Displays the given message on the console.
Compile and Run
Save the program as:
HelloWorld.java
Compile:
javac HelloWorld.java
This produces:
HelloWorld.class
Run:
java HelloWorld
Output:
Hello World
Complete Flow
HelloWorld.java
    ↓
  javac compiler
    ↓
HelloWorld.class
  (Bytecode)
    ↓
    JVM
    ↓
Hello World
```

## Area | Example

Enterprise Applications | ERP, HR, payroll, banking systems

Web Applications | E-commerce websites

Mobile Applications | Android applications

Cloud Applications | Cloud backend services

Big Data | Hadoop, Apache Spark

Microservices | Payment and order services

Desktop Applications | Student Management System

Web Services / APIs | REST APIs

Messaging Systems | Apache Kafka

Games | Minecraft Java Edition

AI/ML | Machine-learning services

IoT / Embedded | Device-management applications